

IDV-ASAS 设计文档

Identity V Area Selection Assistance System 《第五人格》区域选择辅助系统

目录

- [1. 第五人格简介](#)
- [2. 项目简介](#)
- [3. 痛点分析](#)
- [4. 场景定制方案](#)
- [5. 系统架构图](#)
- [6. 技术选型](#)
- [7. 核心模块设计](#)
- [8. 状态机流程](#)
- [9. 数据资源体系](#)
- [10. 错误处理与容错机制](#)
- [11. 历史会话管理](#)
- [12. 总结](#)

1. 第五人格简介

《第五人格》（Identity V）是由网易游戏（NetEase）于 2018 年推出的一款**非对称性对抗竞技手游**（Asymmetrical Battle Arena）。与传统对称性竞技游戏（如 MOBA 类的 5v5 对抗）不同，《第五人格》的对局双方人数不对等、目标不一致：一方为 1 名监管者（Hunter），另一方为 4 名求生者（Survivor），双方围绕密码机破译、密码门逃生与追击淘汰展开博弈。

1.1 核心玩法

一局完整的排位赛流程如下：

- Ban/Pick（禁选环节）**：双方轮流禁用并选择角色。监管者选定 1 名角色，求生者阵营选定 4 名互不相同的角色。
- 区域选择**：Ban/Pick 结束后，排位地图被划分为 9~12 个固定区域。求生者与监管者双方依次自主选择区域，所选区域内一定范围随机出生。此环节是双方争胜博弈的关键起点。
- 对局进行**：求生者需破译 5 台密码机以开启密码门，至少 3 名求生者逃出大门方为胜利；监管者则需淘汰（放飞或淘汰）足够多的求生者以阻止逃生。
- 结算**：根据逃生人数判定胜负结果及排位积分变动。

1.2 求生者阵营概述

求生者阵营目前拥有 51 名角色（截至 2026 年），每名角色拥有独特的技能机制与六维属性：

属性维度	说明
牵制能力	被监管者追击时的周旋能力，即"一溜"能力
破译能力	破译密码机的速度，直接缩短对局时长
运营能力	后期运营阶段的综合表现
救援能力	营救被挂上狂欢之椅队友的能力
辅助能力	不在场时通过道具远程辅助队友的能力
ob 能力	在场时辅助被追击队友的能力（需放弃破译）

1.3 天赋系统

每位求生者可携带 2 个大天赋，从以下 4 种中选择：

天赋	类型	作用
回光返照	运营天赋	激活后大幅提升破译与开门速度
化险为夷	运营天赋	救人后为被救者提供短暂无敌，拖延时间
膝跳反射	牵制天赋	翻越板窗后获得加速，利于板窗区牵制
飞轮效应	牵制天赋	主动技能，提供位移与短暂减伤，兼顾救援与 ob

1.4 区域选择的战略意义

区域选择是 Ban/Pick 之后的第一个关键决策点。选点的好坏直接决定了：

- **抗压位**的部署：牵制能力强的角色占据危险点位抗压，为队友争取破译时间。
- **安全位**的保障：牵制弱但破译强的角色放置在安全点位，保障稳定输出。
- **联动配合**：有联动关系的角色放在邻近点位，发挥组合优势。
- **天赋适配**：携带牵制天赋的角色可以占据更危险的点位，携带运营天赋的角色则应放在安全位。

排位地图共 9 张：军工厂、圣心医院、红教堂、湖景村、月亮河公园、里奥的回忆、永眠镇、唐人街、不归林。每张地图的选点编号、俗称、板窗资源、牵制难易程度各不相同，需要针对性地分析。

2. 项目简介

2.1 项目定位

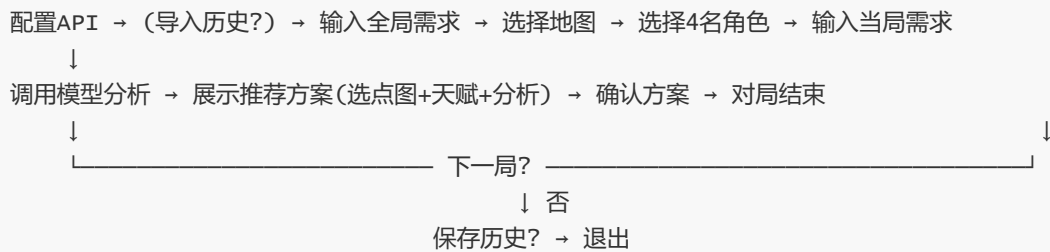
IDV-ASAS（Identity V Area Selection Assistance System，第五人格区域选择辅助系统）是一个**基于大语言模型的桌面 GUI 应用**，旨在为《第五人格》排位赛中的**求生者阵营**提供**天赋选择与区域选择推荐方案**。

玩家在排位 Ban/Pick 结束后，输入当前阵容（4 名角色）与所选地图，IDV-ASAS 会综合角色六维属性、天赋适配性、地图选点特性、角色联动关系等数据，调用 OpenAI 兼容的大语言模型生成 2~3 套合理的"天赋 + 选点"推荐方案，并通过可视化界面直观展示给玩家。

2.2 设计目标

目标	说明
专业	接入完整的角色与地图数据库，综合天赋、选点、联动三维度分析，超越官方仅基于角色阵容的推荐
直观	图形化展示推荐方案：地图选点图上叠加角色头像、天赋图标并配以文字分析
灵活	支持用户自定义全局需求与当局需求，模型优先服从用户意图
可持续	支持历史会话保存与导入，多局连续分析时自动继承上下文
兼容	兼容任意 OpenAI 接口的模型服务（OpenAI、DeepSeek、SiliconFlow 等）

2.3 使用流程概览



2.4 项目结构总览

```

IDV-ASAS/
├── AGENTS.md # 顶层使用流程与实现指南
├── DICTIONARY.md # 角色/地图/天赋中英文字典
├── overview.md # 项目概述
├── DESIGN-DOC.md # 本设计文档
├── app/ # 应用程序主体
│   ├── Cargo.toml # Rust 依赖声明
│   ├── assets/Deng.ttf # 内嵌中文字体
│   ├── config.json # 运行时 API 配置
│   ├── history/ # 会话历史 JSON 文件
│   ├── src/ # Rust 源代码
│   ├── DESIGN.md # 实现原理文档
│   └── USAGE.md # 用户使用说明
├── resources/ # 后端数据资料
│   ├── AGENTS.md # 后端流程说明
│   ├── README.md # 求生者属性与天赋选择总纲
│   ├── characters/ # 51 名角色数据 (*.md)
│   └── maps/ # 9 张地图数据 (*.md)
└── icons/ # 前端图标资源
    ├── AGENTS.md # 前端流程说明
    ├── maps/ # 地图概念图 (PNG)
    ├── area-selection/ # 区域选点底图 (PNG)
    ├── characters/ # 角色头像 (PNG, 120x120)
    ├── personas/ # 天赋图标 (PNG)
    └── positions/ # 选点坐标数据 (*.md)

```

2.5 核心特性

- 1. **天赋-选点联合推荐**：不止推荐选点，还推荐每个角色的天赋搭配。天赋影响选点策略（牵制天赋可占危险点位，运营天赋需放安全位），两者联合分析才能给出合理方案。
- 2. **队伍联动分析**：将角色间的联动关系（如医生-咒术师、小女孩-野人等）纳入考量，联动方放在邻近点位以发挥组合优势。
- 3. **多方案推荐**：一次给出 2~3 套方案，按推荐程度排序，玩家可翻页浏览并选择最适方案。
- 4. **可视化展示**：在地图选点底图上按推荐位置叠加角色头像，右侧展示天赋图标，下方展示文字分析，直观清晰。
- 5. **可打断调用**：模型分析过程中支持随时打断，避免网络异常或模型卡死时无限等待。
- 6. **历史续局**：保存会话历史，导入后自动继承全局需求与累计统计，历史局以紧凑形式进入上下文避免 Token 膨胀。

3. 痛点分析

3.1 痛点一：官方推荐未考虑天赋选择

《第五人格》官方于 2026 年 7 月 2 日正式服更新中增添了基于求生者阵容及角色特性的区域选择推荐。然而，该推荐**仅基于角色阵容**，未考虑天赋选择对选点的影响。

求生者的区域选择与天赋选择息息相关。同一角色，携带不同天赋时其选点策略应截然不同：

天赋配置	选点策略
双牵制天赋（膝跳反射 + 飞轮效应）	可占据危险点位抗压牵制
运营天赋（回光返照 + 化险为夷）	应放在安全点位，避免无牵制天赋时被迫牵制

例如，医生携带"膝跳反射 + 飞轮效应"时可以放在危险点位抗压；而携带"回光返照 + 化险为夷"时则必须放在安全点，因为医生本身牵制能力仅"中等"，没有牵制天赋补足时无法承担抗压任务。

官方推荐无法做到这种**天赋-选点联合分析**，导致推荐方案在实际使用中常常出现"天赋与选点不匹配"的问题。

3.2 痛点二：官方推荐缺乏队伍整体联动

官方在给出区域选择推荐时，是对求生者队伍中**每一位玩家单独**给出的。这意味着推荐仅仅停留在"该角色是否适合该点位"层面，而忽略了求生者整体的配合性与联动性。

然而，角色间的联动关系是求生者阵营的上限所在。例如：

联动组合	联动要点
医生 + 咒术师	医生不可被首追，需放在安全点
小女孩 + 野人	可在邻近点位发挥组合优势
作曲家 + 啦啦队员	配合破译与激励加成
医生 + 击球手	医生不可被首追，需放在安全点

如果仅单独推荐，无法将有联动的角色放在邻近的、利于配合的点位。此外，每位玩家只看到自己的推荐，对队伍整体缺乏认识，无法理解为什么自己被放在这个位置。

3.3 痛点三：通用 Agent 不理解游戏机制

通用大语言模型（如 ChatGPT、DeepSeek 等）直接询问"第五人格选点推荐"时，存在以下问题：

- 1. **版本滞后**：模型训练数据可能不包含最新版本的角色与地图信息。
- 2. **不理解天赋-选点关系**：模型缺乏对天赋如何影响选点策略的专业认知。
- 3. **不理解选点俗称**：玩家常说"小树林""沙包"等俗称，模型无法与编号对应。
- 4. **不了解选点禁区与必选区**：某些地图点位无论如何不可选（如军工厂的 1、3、4 号点），某些点位必须选（如军工厂的 5 号点），通用模型不了解这些约束。
- 5. **不了解联动关系**：角色间的特定联动配合关系需要专门的数据库支撑。

3.4 痛点四：玩家经验难以系统化

区域选择已上线 4 年有余，网络上已有不少高手玩家的经验之谈。但这些经验散落在各类攻略帖、视频与社区讨论中，缺乏系统化整理，普通玩家难以学习和应用。

玩家在面对区域选择时，往往只能凭直觉或有限的攻略知识做决策，容易因选点不当而葬送对局——这也正是 2026 年 4 月 28 日卧谈会上玩家反馈的核心问题。

4. 场景定制方案

4.1 整体方案

IDV-ASAS 针对上述痛点，设计了以下场景定制方案：

IDV-ASAS 场景定制方案			
① 角色数据库	→	六维属性、天赋推荐、地图适配、联动关系	
② 地图数据库	→	选点俗称、板窗资源、牵制难易、选点模式	
③ 天赋总纲	→	天赋说明、选择一般规律	
④ 经验策略库	→	整合高手经验为决策策略	
⑤ 大语言模型	→	综合以上数据，生成天赋+选点联合推荐方案	
⑥ 用户需求	→	全局需求 + 当局需求，优先于数据	
输出：2~3 套方案（每套含天赋选择 + 区域选择 + 分析说明）			

4.2 天赋-选点联合推荐

解决痛点一：将天赋选择纳入选点分析。

具体流程：

- 1. **读取角色六维**：根据当前 4 名角色，比对各自的牵制能力、破译能力、运营能力、救援能力、辅助能力与 ob 能力。

2. **生成天赋分配**：根据队伍整体属性（牵制强则多带运营天赋，牵制弱则补牵制天赋），给出 2~3 组合理的天赋分配。约束条件：
- 每角色 2 个天赋
 - 通常 1~3 个化险为夷，0~4 个回光返照
 - 牵制弱的角色需带牵制天赋，除非放在极安全点位
 - 需要救人的角色可考虑带飞轮效应辅助救援
 - 选点板窗多时可膝跳反射替代飞轮效应
3. **为每组天赋分配选点**：考虑天赋对选点的影响——携带牵制天赋的角色可放危险点位抗压，携带运营天赋的角色应放安全点位。
4. **综合考虑对局情况**："闷搏命"（监管者不追击抗压位而是追击安全位）、"换追"、"干扰密码机"等可能情况，力求给出当前天赋下的优势选点。

4.3 队伍联动分析

解决痛点二：将有联动的角色整体考虑。

IDV-ASAS 接入了角色联动关系数据库（存放于 `resources/characters/*.md` 的"联动与配合"板块中）。例如医生的角色数据中明确记录：

医生与咒术师体系：医生不可被首追，需要放在安全点。医生与击球手体系：医生不可被首追，需要放在安全点。医生与小说家体系：医生不可被首追，需要放在安全点。

在进行选点推荐时，系统会将联动方整体考虑，放在邻近的、利于配合的点位。

此外，为让每位玩家对队伍有整体认识，IDV-ASAS 在给出推荐时对**每一位玩家展示所有求生者的天赋推荐和选点推荐**，而非仅展示各自的推荐。

4.4 专属数据库支撑

解决痛点三：构建专属数据库，让模型理解游戏机制。

IDV-ASAS 构建了完整的结构化数据库，作为模型 system prompt 注入：

数据源	内容	注入方式
<code>DICTIONARY.md</code>	51 名角色 + 9 张地图 + 4 种天赋的中英文名映射	system prompt
<code>resources/README.md</code>	求生者属性构成、天赋选择与区域选择一般规律	system prompt
<code>resources/characters/README.md</code>	天赋说明、角色六维数据格式、联动与配合说明	system prompt
<code>resources/maps/README.md</code>	地图选点说明、选点俗称、牵制难易、选点推荐模式	system prompt
<code>resources/characters/*.md</code>	各角色六维数据、天赋推荐、地图适配、联动关系	user prompt（每局动态）
<code>resources/maps/*.md</code>	各地图选点编号、俗称、板窗资源、选点模式	user prompt（每局动态）

通过这种方式，模型获得了完整的游戏知识，能够理解"小树林"等俗称、遵守选点禁区约束、考虑角色联动关系。

4.5 经验策略整合

解决痛点四：整合高手经验为决策策略。

地图数据文件中已整合了高手玩家的选点经验，例如军工厂数据中明确列出了三种选点模式：

选点模式	选取区域	特点
L 型选点	2、5、8、9 号点	破译速度最低，但 9 号大房极安全
L 型变体选点	2、5、7、9 号点	破译速度中等，压力位为 2、7 号
T 型/十字选点	2、5、6、8 号点	破译速度最高，三人分摊压力

并明确了禁区（1、3、4 号点不可选）和必选区（5 号点必选），以及各点位的角色放置建议。模型在分析时需遵守这些约束——除非用户特别声明。

4.6 用户需求优先

所有推荐方案均以用户需求为最高优先级。用户需求分为两层：

- **全局需求**：对会话中每一局均生效（如"队伍需要携带两个化险为夷"）
- **当局需求**：仅对当前局生效（如"本局医生想打野，不要放安全点"）

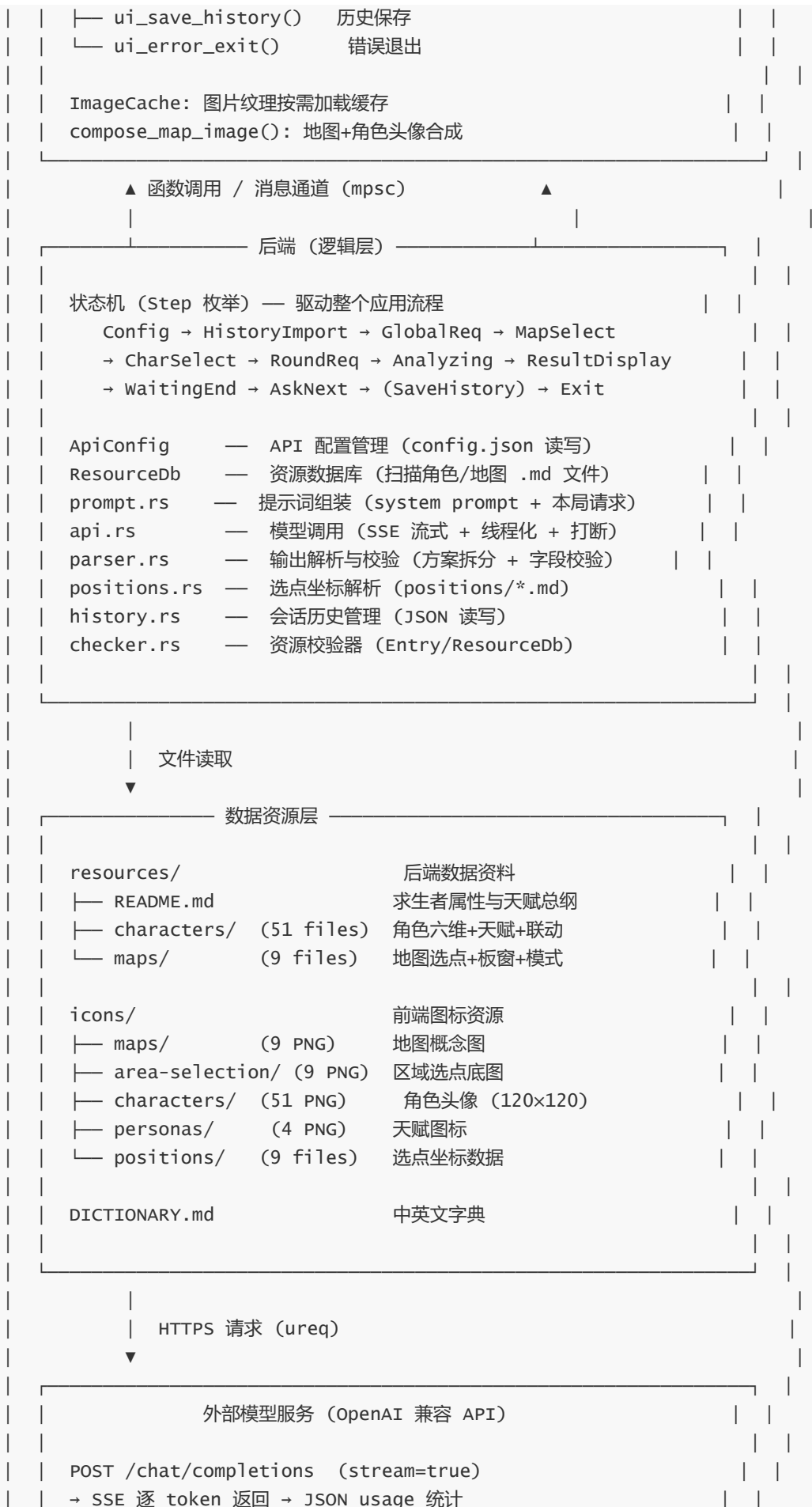
当用户需求与数据库推荐有冲突时，以用户需求为准。

5. 系统架构图

5.1 整体架构

IDV-ASAS 采用**单进程前后端一体化**架构。虽然分为前端与后端两部分，但实质上是同一个应用程序内部的模块划分，通过函数调用与消息通道通信，无需联网。





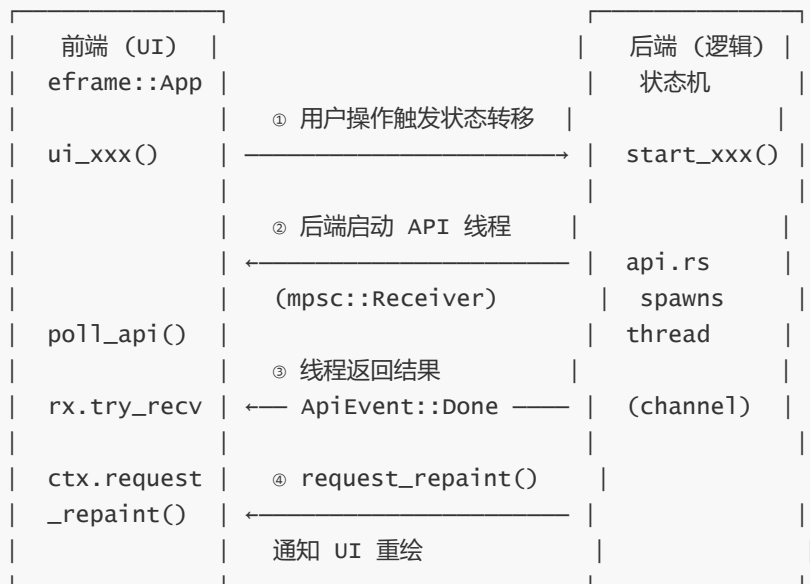
支持: OpenAI / DeepSeek / siliconFlow / 自建网关 等

运行时产物:

└─ app/config.json API 配置 (明文)
└─ app/history/*.json 会话历史

5.2 前后端通信模型

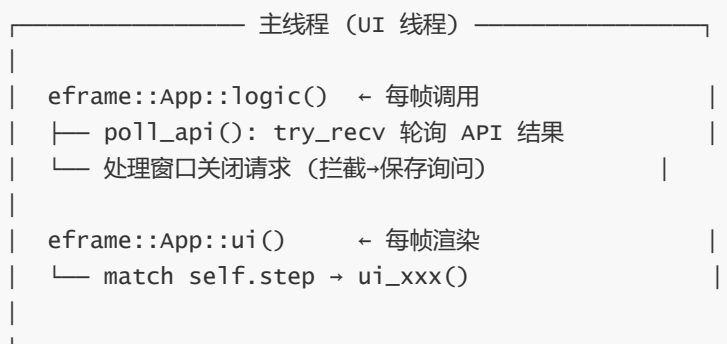
虽然项目中"前端"与"后端"是同一进程内的模块,但它们之间的通信模式值得说明:



关键设计:

- **非阻塞**: API 调用在独立线程执行, UI 线程不阻塞, 保持响应性。
- **消息驱动**: 通过 `mpsc::channel` 传递 `ApiEvent`, UI 线程在每帧 `logic()` 中 `try_recv()` 轮询。
- **重绘机制**: API 线程完成时通过 `ctx.request_repaint()` 通知 UI 重绘, 避免空转。
- **打断机制**: UI 线程通过 `Arc<AtomicBool>` 共享标记通知 API 线程停止。

5.3 线程模型



▲



6. 技术选型

6.1 选型总览

层面	技术	版本	选型理由
编程语言	Rust	1.75+	后端必须使用 Rust（resources/AGENTS.md 要求）；内存安全、零成本抽象
GUI 框架	eframe + egui	0.36	纯 Rust 桌面 GUI，即时模式渲染，跨平台
渲染后端	wgpu	30	Vulkan + GL 后端，eframe wgpu 配置
窗口后端	X11 only	—	eframe features 仅启用 x11，禁用 wayland 和 accesskit
图片解码	image	0.25	仅启用 PNG 解码，解码角色头像/地图/天赋图标
HTTP 客户端	ureq	3.4	轻量同步 HTTP 客户端，用于 OpenAI 兼容 SSE 流式 API 调用
序列化	serde + serde_json	1	配置与历史记录的 JSON 读写
时间处理	chrono	0.4	历史记录时间戳命名（ %Y%m%d-%H%M%S ）
中文字体	内嵌 Deng.ttf	—	等线字体，编译时 <code>include_bytes!</code> 内嵌

6.2 选型详细说明

6.2.1 为什么选择 Rust + eframe/egui

考量维度	说明
后端硬性要求	<code>resources/AGENTS.md</code> 明确要求"后端必须使用 Rust 完成"
前端灵活性	<code>icons/AGENTS.md</code> 指出前端"可以不使用 rust 语言实现"
一体化优势	后端用 Rust，前端也用 Rust（eframe/egui），可统一前后端为一个进程，无需进程间通信
egui 特性	即时模式 GUI，天然适合状态机驱动；内置图片纹理管理；wgpu 渲染后端性能足够
部署便利	纯 Rust 编译为单一可执行文件，无运行时依赖（除 X11 库），部署简单

注：前端设计文档 `icons/AGENTS.md` 建议采用 2560×1440 窗口化显示，但实际实现（`app/DESIGN.md` 与 `app/src/main.rs`）采用 1280×720。这是因为项目尚处于测试阶段，1280×720 是更通用的最小可用分辨率。

6.2.2 为什么选择 ureq 而非 request

维度	ureq	request
依赖体积	轻量，依赖少	依赖 tokio 全家桶，编译时间长
异步模型	同步 API	异步 tokio
适用性	本项目 SSE 读取为同步循环，与 ureq 天然匹配	异步增加复杂度

ureq 3.4 的 `http_status_as_error(false)` 配置允许我们手动处理 HTTP 状态码，方便向用户展示具体错误信息。

6.2.3 渲染后端选择

```
// main.rs
let mut wgpu_setup = eframe::egui_wgpu::WgpuSetupCreateNew::without_display_handle();
wgpu_setup.instance_descriptor.backends = wgpu::Backends::VULKAN | wgpu::Backends::GL;
```

选择 `VULKAN` | `GL` 双后端的原因：

- **Vulkan**：现代图形 API，性能最佳，Linux 桌面通常支持。
- **GL (OpenGL)**：作为后备，兼容不支持 Vulkan 的环境。
- **禁用 Wayland**：Wayland 在某些环境下存在输入法兼容问题，强制 X11 更稳定。

6.2.4 图片处理方案

项目涉及大量图片资源（地图概念图、选点底图、51 个角色头像、4 个天赋图标），采用两层缓存：



区域选点图的合成在内存中完成（RGBA 像素级 alpha 混合），而非 painter 图层叠加，避免了渲染顺序问题：

```
选点底图(RGBA) + 角色头像(scaled) → alpha混合 → 合成图 → TextureHandle → egui::Image
```

6.2.5 OpenAI 兼容 API 通信

IDV-ASAS 兼容任意 OpenAI 接口的服务商：

服务商	Base URL	模型示例
OpenAI	<code>https://api.openai.com/v1</code>	<code>gpt-4o-mini</code>
DeepSeek	<code>https://api.deepseek.com/v1</code>	<code>deepseek-chat</code>
SiliconFlow	<code>https://api.siliconflow.cn/v1</code>	<code>deepseek-ai/DeepSeek-V3</code>
自建网关	<code>http://localhost:8080/v1</code>	自定义

通信流程：

1. 构建 JSON body: { model, messages, temperature, stream: true, stream_options: { include_usage: true } }
2. POST → endpoint (自动补全 /chat/completions)
3. Header: Authorization: Bearer <api_key>
4. 响应: SSE text/event-stream
 - └─ data: {"choices":[{"delta":{"content":"..."}}]}
 - └─ data: {"usage":{"prompt_tokens":N,"completion_tokens":M}} (末尾)
 - └─ data: [DONE]

兼容性处理: 若服务不支持 `stream_options`, 自动降级重试 (`include_usage=false`), 此时 Token 统计按 字符数 × 0.75 估算。

7. 核心模块设计

7.1 模块概览

app/src/	
└─ main.rs	程序入口: 创建窗口, 配置 wgpu 渲染
└─ app.rs	核心结构体 IdvApp + 状态机 + eframe::App trait 实现
└─ ui_impl.rs	全部 UI 渲染方法 (12 个步骤界面) + 图片合成函数
└─ config.rs	API 配置 (ApiConfig): 加载/保存/端点规范化
└─ checker.rs	资源校验器 (ResourceDb): 扫描地图与角色数据文件
└─ positions.rs	选点坐标解析器 (MapPosition): 解析 positions/*.md
└─ prompt.rs	提示词组装: system prompt + 本局请求 (完整版/紧凑版)
└─ history.rs	会话历史管理 (Session): 保存/加载/列表/命名
└─ api.rs	模型调用: SSE 流式读取 + 打断 + Token 统计
└─ parser.rs	输出解析器: 方案拆分 + 字段校验 + 描述截断
└─ images.rs	图片纹理缓存 (ImageCache): 按需加载 PNG → TextureHandle

7.2 提示词组装模块 (prompt.rs)

设计理念

将静态资料与动态数据分离注入:

注入类型	时机	内容	生命周期
System Prompt	启动时一次性组装	后端流程说明 + 字典 + 通用资料 + 输出格式要求	会话级常量
Full Request	每局动态组装	紧凑信息 + 本局地图完整数据 + 四名角色完整数据	每局重新生成
Compact Request	历史局上下文	精简的本局摘要 (省略大段资料)	续局时用于历史消息对

System Prompt 组成

```

system_prompt = format!(
    "你是 IDV-ASAS 分析核心\n
    # 你的任务\n{agents_md}\n          // resources/AGENTS.md\n
    # 输出要求\n{格式规范}\n          // 方案格式、选点行格式、描述要求\n
    # 角色与地图字典\n{dictionary}\n    // DICTIONARY.md\n
    # 数据库通用资料\n
    ## 求生者与队伍总纲\n{readme}\n    // resources/README.md\n
    ## 天赋与角色说明\n{chars_readme}\n // resources/characters/README.md\n
    ## 地图与选点说明\n{maps_readme}\n // resources/maps/README.md\n
    # 重要覆盖说明\n{编号vs俗称}"
)

```

本局请求组装

```

full_request = format!(
    "【第 N 局】\n
    地图: {地图中文名}\n
    求生者: {角色1}、{角色2}、{角色3}、{角色4}\n
    全局需求: {全局需求}\n
    本局需求: {本局需求}\n
    (以上为本局全部输入。此前的历史局记录见对话上文...) \n\n
    【本局地图资料: {地图中文名}】\n
    {地图数据文件内容}\n
    {角色1数据}\n
    {角色2数据}\n
    {角色3数据}\n
    {角色4数据}\n\n
    请结合以上资料与全部需求, 给出 2~3 套天赋选择与区域选择方案。"
)

```

紧凑请求 (历史局上下文)

当导入历史会话续局时, 历史局以紧凑形式进入上下文, 省略大段资料以避免 Token 膨胀:

```

compact_request = format!(
    "【第 N 局】\n
    地图: {地图中文名}\n
    求生者: {角色1}、{角色2}、{角色3}、{角色4}\n
    全局需求: {全局需求}\n
    本局需求: {本局需求}\n
    (该局地图与角色详细资料此处从略)"
)

```

7.3 模型调用模块 (api.rs)

调用流程

```

start_api_call()
|
├─ 创建 mpsc::channel (ApiEvent)
├─ 创建 Arc<AtomicBool> (interrupt flag)

```

```

└─ thread::spawn → chat_stream_threaded()
  │
  │   └─ attempt() (include_usage=true)
  │     │   └─ 构建 JSON body
  │     │   └─ ureq POST → SSE 响应
  │     │   └─ 读取循环 (BufReader + sync_channel)
  │     │     │   └─ recv_timeout(150ms)
  │     │     │   └─ 检查 interrupt flag
  │     │     │   └─ 120s 超时退出
  │     │     └─ 解析 SSE delta content + usage
  │     └─ 返回 StreamOutcome { content, usage, interrupted, elapsed }
  │
  │   └─ 若 stream_options 报错 → attempt() (include_usage=false)
  └─ tx.send(ApiEvent::Done(result))
└─ UI 线程 poll_api() 每帧 try_recv()
  │   └─ Empty → request_repaint()
  │   └─ Done(Ok) → 解析校验 → ResultDisplay 或 重试
  └─ Done(Err) → ErrorExit

```

打断机制

```

// UI 线程
fn do_interrupt(&mut self) {
    if let Some(flag) = &self.api_interrupt {
        flag.store(true, Ordering::Relaxed);
    }
}

// API 线程读取循环中
if interrupt.load(Ordering::Relaxed) {
    interrupted = true;
    break 'outer;
}

```

UI 线程设置标记后，API 线程在下次 150ms 轮询循环中检测到并立即停止读取 SSE，返回 `interrupted=true`。

Token 统计策略

数据来源	条件	精度
API 返回 <code>usage</code> 字段	服务支持 <code>stream_options</code>	精确值
字符数估算	服务不支持或降级重试	<code>chars × 0.75</code> 估算，标注"按字符估算"

费用计算：`prompt_tokens / 1M × price_input + completion_tokens / 1M × price_output`

7.4 输出解析与校验模块 (parser.rs)

三阶段处理流程

原始文本

|

▼

阶段一: `split_plans()` — 方案拆分

|

| 以 "方案" 开头的行作为分隔标记

|

| 跳过前言 ("方案" 之前的内容)

|

| 返回 `Vec<String>` (每个方案文本)

▼

阶段二: `parse_single_plan()` — 逐方案解析

|

| 遍历每行:

|

| |— 跳过 "方案" 开头的行

|

| |— "具体描述及分析: " → 提取描述

|

| |— "x 号选点: 角色名: 天赋1、天赋2" → 提取 `Selection`

|

| 返回 `Plan { selections, description }`

▼

阶段三: 合法性校验

|

| |— 角色名存在 (`db.find_character`)

|

| |— 天赋名合法 (4 种)

|

| |— 每角色天赋数 = 2

|

| |— 选点编号 ≤ `max_point` 且 ≠ 0

|

| |— 选点不重复

|

| |— 角色数 = 4

|

| 描述 > 100 字 → 截断至最后一个完整句子 + "..."

▼

`Vec<Plan>` (有效方案列表)

数据结构

```
struct Plan {
    index: usize,           // 方案序号
    selections: Vec<Selection>, // 4 个选点
    description: String,     // 不超过 100 字的分析
}

struct Selection {
    point: u32,             // 选点编号
    character: String,      // 角色中文名
    talents: Vec<String>,   // 2 个天赋中文名
}
```

重试机制

当校验失败时:

1. 将模型输出追加为 assistant 消息
2. 追加错误说明为 user 消息 (含格式要求强调)
3. 重新调用模型
4. 最多重试 3 次
5. 超限则报告"您的模型与该项目不适配", 进入 `ErrorExit`

7.5 图片加载与合成 (images.rs + ui_impl.rs)

ImageCache 设计

```
struct ImageCache {  
    base: PathBuf, // 图片目录  
    textures: HashMap<String, TextureHandle>, // 按文件名缓存  
}
```

首次请求时解码 PNG → 转 RGBA → 创建 TextureHandle 存入缓存；后续直接返回缓存引用。共 4 个 ImageCache 实例：角色头像、地图概念图、选点底图、天赋图标。

区域选点图合成流程

```
compose_map_image(plan, map_stem)  
|  
|— 加载选点底图 PNG → RGBA 缓冲区  
|— 读取 positions/{map}.md → MapPosition  
|   |— map_width, map_height  
|   |— points: Vec<(编号, x, y)>  
|   |— icon_size: u32  
|  
|— 对 plan 中每个 selection:  
|   |— 查找角色英文名 (talent_stem / DICTIONARY)  
|   |— 加载角色头像 PNG → RGBA  
|   |— 缩放至 icon_size × icon_size  
|   |— alpha 混合叠加到地图 (point_center(x,y) 为锚点)  
|  
|— 合成图 → egui::ColorImage → TextureHandle  
|— 返回 → egui::Image 显示
```

所有合成在内存中完成，不依赖 painter 图层叠加，避免渲染顺序问题。

7.6 选点坐标解析 (positions.rs)

选点坐标数据存放于 `icons/positions/*.md`，格式示例：

军工厂

军工厂地图为 `$699 \times 600$` 像素。共有 9 个选点。

1 号选点中心: `$(134,119)$`。

2 号选点中心: `$(345,119)$`。

...

所用角色图标为 `$100 \times 100$` 像素。

解析逻辑：

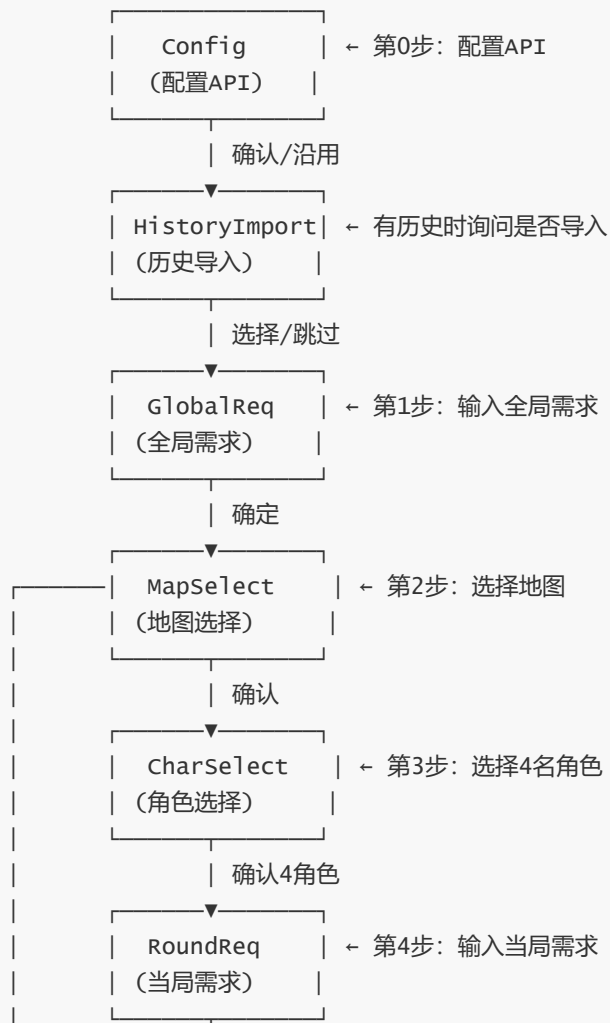
- 识别 `##` 行提取地图中文名
- 识别含"像素"和 `×` 或 `\times` 的行解析地图尺寸与图标尺寸
- 识别含"号选点中心"的行解析选点编号与坐标 `$(x,y)$`
- LaTeX `\times` 需替换为 Unicode `×` 再分割

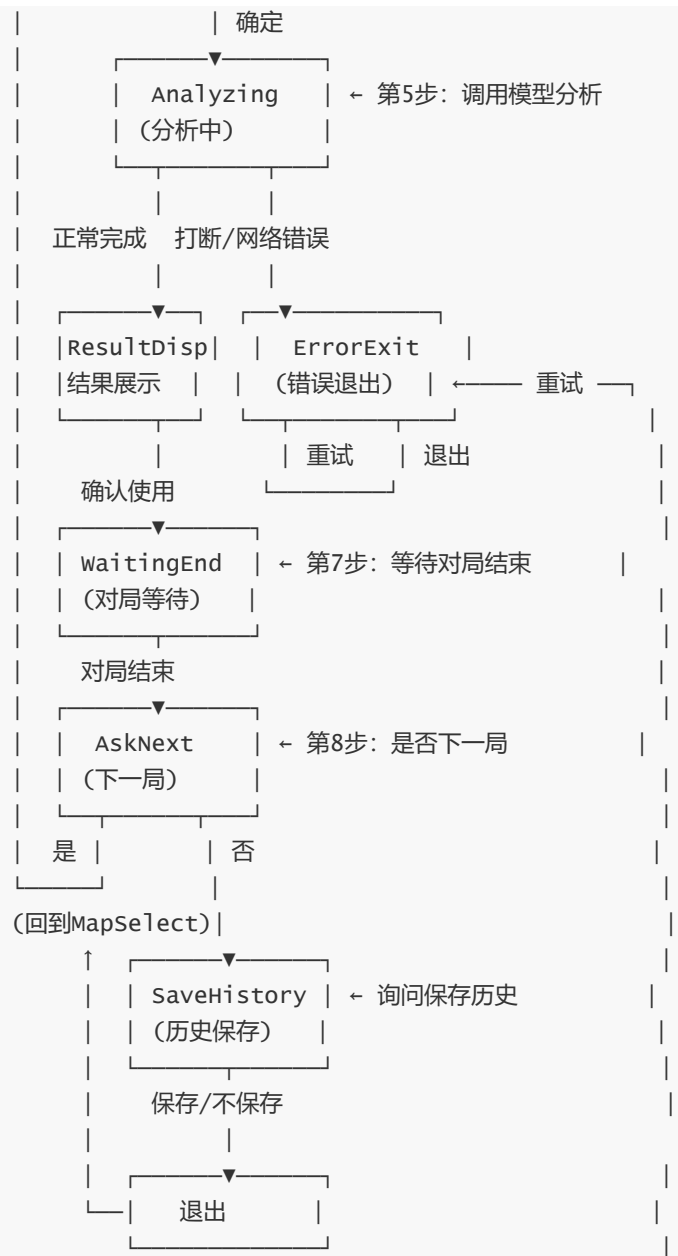
8. 状态机流程

8.1 状态枚举

```
enum Step {  
    Config,           // 第0步: 配置API  
    HistoryImport,    // 历史导入  
    GlobalReq,        // 第1步: 全局需求  
    MapSelect,        // 第2步: 地图选择  
    CharSelect,       // 第3步: 角色选择  
    RoundReq,         // 第4步: 当局需求  
    Analyzing,        // 第5步: 模型分析中  
    ResultDisplay,    // 第6步: 结果展示  
    waitingEnd,       // 第7步: 对局等待  
    AskNext,          // 第8步: 下一局询问  
    SaveHistory,      // 历史保存  
    ErrorExit,        // 错误退出  
}
```

8.2 状态转移图





8.3 关键状态说明

Config (配置 API)

- **有默认配置**：显示当前配置摘要，询问是否沿用。选择"是"直接进入下一步；选择"否"展开编辑表单。
- **无默认配置**：直接展开编辑表单，依次填写 Base URL、API Key、模型名、温度、max_tokens、输入/输出单价。
- 配置保存至 `app/config.json` (明文，需提示安全风险)。

Analyzing (分析中)

- UI 线程在 `logic()` 中每帧调用 `poll_api()` 轮询 API 结果。
- 用户可随时点击"打断"按钮 → `do_interrupt()` → API 线程停止 → `interrupted=true` → 进入 `ErrorExit`。
- 正常完成 → 解析校验 → 成功则进入 `ResultDisplay`，失败则重试 (最多 3 次)。

ResultDisplay (结果展示)

- 左侧：合成后的区域选点图（角色头像叠加在选点位置上）
- 右侧上：天赋选择（4 行角色头像 + 天赋图标）
- 右侧中：分析说明（不超过 100 字）
- 右侧下：翻页按钮（◀▶）+ 确认使用按钮
- 用户翻页浏览多套方案，确认后进入 WaitingEnd

WaitingEnd (对局等待)

- 界面锁定在已确认方案展示页面
- "对局结束"按钮替换"确认使用"按钮
- 点击后显示本局 Token 用量与费用 + 会话累计统计

8.4 窗口关闭拦截

```
fn logic(&mut self, ctx: &egui::Context, _frame: &mut eframe::Frame) {
    // ... poll_api ...
    if ctx.input(|i| i.viewport().close_requested()) {
        if !self.session.rounds.is_empty() && !self.show_save_dialog {
            ctx.send_viewport_cmd(egui::ViewportCommand::CancelClose);
            self.show_save_dialog = true;
            self.step = Step::SaveHistory;
        }
    }
}
```

当用户尝试关闭窗口时，若有未保存的对局记录，程序会拦截关闭请求，跳转到历史保存界面。

9. 数据资源体系

9.1 资源目录结构

IDV-ASAS/	
— DICTIONARY.md	中英文字典（角色/地图/天赋）
— resources/	后端数据资料
— AGENTS.md	后端流程说明（注入 system prompt）
— README.md	求生者属性与天赋选择总纲（注入 system prompt）
— characters/	
— README.md	天赋说明与角色数据格式（注入 system prompt）
— doctor.md	医生：六维+天赋推荐+联动
— mercenary.md	佣兵
— ...（共 51 个角色）	每个角色一个 .md 文件
— lucky-guy.md	幸运儿
— maps/	
— README.md	地图选点说明（注入 system prompt）
— arms-factory.md	军工厂
— ...（共 9 张地图）	每张地图一个 .md 文件

```

|       └─ darkwoods.md          不归林
|
|─ icons/                        前端图标资源
|   │─ AGENTS.md                前端流程说明
|   │─ maps/                    地图概念图 (9 PNG)
|   │   │─ arms-factory.png
|   │   └─ ...
|   │─ area-selection/          区域选点底图 (9 PNG)
|   │   │─ arms-factory.png
|   │   └─ ...
|   │─ characters/              角色头像 (51 PNG, 120×120)
|   │   │─ doctor.png
|   │   └─ ...
|   │─ personas/                天赋图标 (4 PNG)
|   │   │─ borrowed-time.png    回光返照
|   │   │─ flywheel-effect.png  飞轮效应
|   │   │─ knee-jerk-reflex.png  膝跳反射
|   │   │─ tide-turner.png      化险为夷
|   └─ positions/               选点坐标数据 (9 个 .md)
|       │─ arms-factory.md
|       └─ ...
|
|─ app/
|   │─ config.json              运行时生成: API 配置
|   │─ history/                 运行时生成: 会话历史 JSON
|   │   │─ 排位记录1.json
|   │   └─ ...

```

9.2 角色数据格式

每个角色数据文件 (`resources/characters/{英文名}.md`) 包含四个板块:

{角色中文名}

六维数据

牵制能力: {极弱/较弱/中等/较强/极强}
 破译能力: {极弱/较弱/中等/较强/极强}
 运营能力: {极弱/较弱/中等/较强/极强}
 救援能力: {极弱/较弱/中等/较强/极强}
 辅助能力: {极弱/较弱/中等/较强/极强}
 ob 能力: {极弱/较弱/中等/较强/极强}

天赋选择推荐

{天赋组合1}: {推荐理由}
 {天赋组合2}: {推荐理由}
 {天赋组合3}: {推荐理由}

角色与地图

{特定角色与特定地图的联动关系, 或"无"}

联动与配合

{特定角色间的联动关系，影响区域选择}

示例（医生 doctor.md）：

牵制能力：中等
运营能力：极强
救援能力：较弱

回光返照、膝跳反射：最常规的选择。
化险为夷、飞轮效应：需要放在较为安全的点位。
膝跳反射、飞轮效应：可以放在危险点位抗压。

医生与咒术师体系：医生不可被首追，需要放在安全点。

9.3 地图数据格式

每张地图数据文件（resources/maps/{英文名}.md）包含以下板块：

{地图中文名}

地图选点基本信息

{地图中文名}共有 {N} 个选点。

选点俗称

- 1 号选点：{俗称1}/{俗称2}。
- 2 号选点：{俗称}。
- ...

选点板窗资源及牵制难易程度

- 1 号选点：{极难/较难/中等/较好/极好}牵制，{是否推荐选择}。牵制时，{板窗资源描述}。
- ...

地图选点推荐

地图禁区：在用户无特别说明情况下，无论如何都不可以选择 {编号} 号选点。
地图必选区：在用户无特别说明情况下，无论如何都必须选择 {编号} 号选点。

该地图共 {N} 种选点推荐：{模式名称列表}。

{选点模式 1 名称}

求生者占据 {编号1}、{编号2}、{编号3}、{编号4} 号选点。
该选点中压力较大的点位为 {编号} 号选点。
{各点位的角色放置建议}

9.4 字典格式

`DICTIONARY.md` 提供中英文名映射，运行时被注入 system prompt:

角色 characters

医生: doctor
律师: lawyer
... (共 51 名角色)

地图 maps

军工厂: arms-factory。
圣心医院: sacred-heart-hospital。
... (共 9 张地图)

天赋 personas

回光返照: borrowed-time。
飞轮效应: flywheel-effect。
膝跳反射: knee-jerk-reflex。
化险为夷: tide-turner。

字典的用途:

1. 模型通过字典查找角色英文名，从而定位数据文件路径
2. 前端通过 `talent_stem()` 函数将天赋中文名映射为英文名，定位天赋图标文件
3. 代码中通过 `CHAR_ORDER` 与 `MAP_ORDER` 常量维持顺序

9.5 资源路径映射

程序运行时以 `CARGO_MANIFEST_DIR` (即 `app/`) 为基准，向上一级定位项目根目录:

```
let app_root = PathBuf::from(env!("CARGO_MANIFEST_DIR"));
let project_root = app_root.parent().unwrap().to_path_buf();
let resources_dir = project_root.join("resources");
let icons_dir = project_root.join("icons");
```


资源	运行时路径	用途
角色数据	<code>../resources/characters/*.md</code>	六维属性、天赋推荐、联动关系
地图数据	<code>../resources/maps/*.md</code>	选点俗称、牵制难易、选点推荐模式
通用资料	<code>../resources/README.md</code> 等	求生者属性构成、天赋选择一般规律
字典	<code>../DICTIONARY.md</code>	中英文名映射
概念图	<code>../icons/maps/*.png</code>	地图选择界面 3×3 网格
选点底图	<code>../icons/area-selection/*.png</code>	结果展示界面的底图
角色头像	<code>../icons/characters/*.png</code>	角色选择与结果展示 (120×120)
天赋图标	<code>../icons/personas/*.png</code>	结果展示的天赋图标
选点坐标	<code>../icons/positions/*.md</code>	各地图选点中心坐标与图标尺寸
配置文件	<code>app/config.json</code>	API 配置 (明文)
历史记录	<code>app/history/*.json</code>	会话历史

9.6 ResourceDb 资源校验器

`checker.rs` 中的 `ResourceDb` 负责启动时扫描资源目录：

```
struct Entry {
    zh: String,      // 中文名 (取自 .md 首行标题)
    stem: String,    // 文件名 (小写英文)
    path: PathBuf,   // 完整路径
}

struct ResourceDb {
    maps: Vec<Entry>,
    survivors: Vec<Entry>,
}
```

扫描逻辑：

- 遍历 `resources/maps/` 和 `resources/characters/` 下的 `.md` 文件
- 跳过 `README.md`
- 从首行 `## {中文名}` 提取中文名
- 按中文名排序
- 提供按中文名或文件名查找的 `find_map()` / `find_character()` 方法

10. 错误处理与容错机制

10.1 错误分类与处理策略

错误类型	触发条件	处理方式	用户可见信息
API 网络错误	连接拒绝、DNS 解析失败、超时	进入 ErrorExit	提供重试 / 退出选项
API 认证错误	HTTP 401	进入 ErrorExit	"API Key 错误或未填写"
API 路径错误	HTTP 404	进入 ErrorExit	"Base URL 填错"
API 参数错误	HTTP 400	进入 ErrorExit	"温度参数非法"等
模型输出格式错误	角色名不存在、天赋不合法等	自动重试（最多 3 次）	不可见（静默重试）
重试超限	连续 3 次格式校验失败	进入 ErrorExit	"您的模型与该项目不匹配"
用户打断	用户点击"打断"按钮	进入 ErrorExit	"分析已打断"
响应超时	120 秒未收到任何输出	进入 ErrorExit	"等待模型响应超时"
API 线程异常断开	channel 断开	进入 ErrorExit	"API 线程异常断开"
资源加载失败	资源目录不存在或为空	启动时 eprintln	标准错误输出
读取选点数据失败	positions/*.md 不存在	回退默认值 max_point=12	不可见

10.2 ErrorExit 界面

ErrorExit 提供三个选项：

△ {错误信息}

当前模型: {model}

当前端点: {base_url}

重试

重新输入

退出

- **重试**：回到 Analyzing，重新调用模型
- **重新输入**：回到 Config，修改 API 配置
- **退出**：若有未保存对局则进入 SaveHistory，否则直接退出

10.3 输出格式校验的容错

模型输出可能出现的格式问题及处理方式：

问题	处理
多余前言 ("好的, 我来分析...")	<code>split_plans()</code> 跳过"方案"之前的所有行
描述超过 100 字	截断至 100 字以内最后一个完整句子 + "..."
角色名不存在	整体重试, 将错误说明追加到消息列表
天赋名不合法	同上
天赋数不为 2	同上
选点编号超出范围	同上
选点编号重复	同上
角色数不足/超过 4	同上

10.4 API 兼容性容错

问题	处理
服务不支持 <code>stream_options</code>	检测错误含 <code>stream_options</code> → 降级重试 <code>include_usage=false</code>
服务不返回 <code>usage</code> 字段	按字符数 × 0.75 估算, 标注"按字符估算"
端点 URL 格式差异	自动补全: 末尾补 <code>/chat/completions</code> ; 已有则不重复
温度精度问题	发送前 <code>round2()</code> 四舍五入至 2 位小数

11. 历史会话管理

11.1 Session 数据结构

```
struct Session {
    version: u32,           // 版本号 (向后兼容)
    created_at_unix: u64,   // 创建时间 (Unix 时间戳)
    base_url: String,       // 使用的 API 端点
    model: String,          // 使用的模型名
    global_req: String,     // 全局用户需求
    rounds: Vec<RoundRecord>, // 各局记录
}

struct RoundRecord {
    index: usize,           // 局序号
    map: String,            // 地图中文名
    survivors: Vec<String>, // 角色中文名列表
    round_req: String,      // 当局需求
    compact_request: String, // 紧凑请求 (用于续局上下文)
    assistant_reply: String, // 模型原始回答
    interrupted: bool,      // 是否被打断
}
```

```
usage: Option<Usage>,      // Token 用量
cost: f64,                 // 费用
estimated: bool,           // 是否为估算值
}
```

11.2 历史保存

触发时机：用户在以下情况退出程序前触发：

- 完成对局后选择"否"不开启下一局
- API 错误导致无法分析，用户不希望重试直接退出
- 用户尝试关闭窗口时拦截

保存流程：

```
询问"是否保存本次会话历史？"
├─ 保存 → 询问"是否命名？"
│   └─ 是 → 用户输入名称 → 以该名称保存
│       └─ 否 → 以当前时间命名（如 20260905-003530）
│           └─ 文件名安全处理：空格→-，过滤特殊字符
└─ 不保存 → 舍弃，直接退出
```

文件位置： `app/history/{名称}.json`

11.3 历史导入

触发时机：第 0 步配置完成后、第 1 步开始前

导入流程：

```
检测 app/history/ 目录下是否存在 .json 文件
├─ 有 → 询问"是否导入历史会话？"
│   └─ 是 → 展示历史列表
│       └─ 列表项显示：模型、局数、累计Token、费用、全局需求摘要
│           └─ 点击列表项选择导入
│               └─ 导入后：全局需求自动继承，直接进入第2步(地图选择)
│                   └─ 顶部提示：✓ 已导入历史会话（共 N 局记录）
└─ 否 → 跳过，开始全新会话
└─ 无 → 直接进入第1步
```

11.4 续局上下文管理

导入历史后继续分析时，历史局以**紧凑形式**进入上下文，避免 Token 膨胀：

```
messages = [
  { role: "system", content: system_prompt },          // 静态资料（一次性注入）

  // 历史局（紧凑形式，省略大段地图与角色资料）
  { role: "user", content: "【第1局】\n地图: 军工厂\n..." }, // compact_request
  { role: "assistant", content: "方案1: \n5号选点: ..." },   // assistant_reply
]
```

```
{ role: "user", content: "【第2局】\n地图: 红教堂\n..." },
{ role: "assistant", content: "方案1: \n..." },

// 当前局 (完整资料)
{ role: "user", content: "【第3局】\n地图: 湖景村\n...
                        【本局地图资料: 湖景村】\n{完整地图数据}
                        【角色资料: 医生】\n{完整角色数据}
                        ..." }, // full_request
]
```

设计要点:

- 历史局只保留紧凑摘要 (地图名、角色名、需求), 省略完整的地图与角色资料
- 模型的历史回答也保留在上下文中, 作为方案参考
- 当前局注入完整的地图与角色数据, 确保分析准确
- 全局需求对每一局均生效, 在 system prompt 和 compact request 中都有体现

11.5 会话累计统计

每局确认完成后, 显示:

本局 Token 用量	
输入: {prompt_tokens}	
输出: {completion_tokens}	
总计: {total_tokens}	
本局费用: ¥{cost}	
{按字符估算 / 精确统计}	
会话累计	
总输入: {累计 prompt_tokens}	
总输出: {累计 completion_tokens}	
总费用: ¥{累计 cost}	

Session::totals() 方法遍历所有 RoundRecord 累加 usage 与 cost。

12. 总结

IDV-ASAS 是一个面向《第五人格》排位赛求生者阵营的**天赋选择与区域选择辅助系统**。它通过以下核心设计解决了玩家在区域选择环节的痛点:

1. **专属数据库**: 构建了 51 名角色的六维属性 + 天赋推荐 + 联动关系数据库, 以及 9 张地图的选点俗称 + 板窗资源 + 牵制难易 + 选点模式数据库, 弥补了通用大语言模型对游戏机制理解不足的缺陷。
2. **天赋-选点联合推荐**: 超越官方仅基于角色阵容的推荐, 将天赋选择对选点策略的影响纳入分析, 给出"天赋 + 选点"联合方案。
3. **队伍联动分析**: 将角色间的联动关系整体考虑, 联动方放在邻近点位, 并为每位玩家展示全队推荐方案。
4. **线程化流式调用**: 采用独立线程进行 SSE 流式 API 调用, UI 不阻塞, 支持随时打断, 并通过 Arc<AtomicBool> 实现跨线程中断信号。

5. **状态机驱动**：11 个状态的有限状态机清晰驱动整个应用流程，从配置到分析到结果展示到历史管理，覆盖完整使用闭环。
6. **历史续局**：紧凑请求机制避免 Token 膨胀，支持多局连续分析且自动继承全局需求与累计统计。
7. **兼容性**：兼容任意 OpenAI 接口的服务商，自动处理端点补全、stream_options 降级、Token 估算等兼容性问题。

该系统采用 Rust + eframe/egui 技术栈实现，单进程前后端一体化架构，编译为单一可执行文件，部署简单，适合桌面端使用。